

Mobile Platform VPN Implementation Research

Mobile Platform VPN Implementation Research

Deep Analysis: Android, iOS, and HarmonyOS VPN Client Development

Research Date: July 2025 **Scope:** Native integration, background execution, battery optimization, cross-platform code sharing strategies **Phase Context:** Flutter and React Native are mature for mobile. Tauri v2 mobile is emerging. HarmonyOS NEXT supports Flutter, React Native, uni-app, Taro, ArkUI-X with varying maturity levels.

Table of Contents

- [1. Android VPN Development](#)
 - [2. Android Background Execution](#)
 - [3. iOS VPN Development](#)
 - [4. iOS Constraints & Considerations](#)
 - [5. HarmonyOS VPN Development](#)
 - [6. HarmonyOS Cross-Platform Strategy](#)
 - [7. Mobile Rust Core Integration](#)
 - [8. Mobile UI Patterns for VPN Apps](#)
 - [9. Push Notifications for VPN](#)
 - [10. Biometric Authentication](#)
 - [11. Mobile Performance Optimization](#)
 - [12. Real-World Mobile VPN App Architectures](#)
 - [13. Platform Comparison Matrix](#)
 - [14. Mobile-Specific Danger Zones](#)
 - [15. Recommended Architecture](#)
-

1. Android VPN Development

1.1 VpnService Architecture

Android VPN development centers on the `VpnService` class, which applications extend to create and manage VPN tunnels. The system provides a `Builder` inner class to configure VPN interface parameters.

"The primary API for implementing your custom tunneling protocols in `NEPacketTunnelProvider`, that allows you to tunnel traffic on an IP layer. They run as app extensions in the background handling network traffic." [^129^]

1.2 Builder Pattern Configuration

The `VpnService.Builder` provides a fluent API for configuring the VPN tunnel:

```
val builder = Builder().apply {
    setSession("MyVPN")
    addAddress("10.0.0.2", 32)
    addDnsServer("8.8.8.8")
    addRoute("0.0.0.0", 0) // Route all traffic
    setMtu(1500)
    // Split tunneling
    addAllowedApplication("com.example.app") // Only this app
                                         uses VPN
    // OR
    addDisallowedApplication("com.example.banking") // This app
                                         bypasses VPN
}
val vpnInterface = builder.establish()
```

Key Builder methods [^25^]:

- `addAddress(String address, int prefixLength)`: Sets the VPN interface IP address
- `addRoute(String address, int prefixLength)`: Routes traffic through the VPN
- `addDnsServer(String address)`: Configures DNS servers for the VPN
- `setMtu(int mtu)`: Sets the Maximum Transmission Unit (default 1500)
- `addAllowedApplication(String packageName)`: Whitelist apps for VPN (mutually exclusive with disallowed)
- `addDisallowedApplication(String packageName)`: Blacklist apps from VPN
- `allowBypass()`: Allows apps to bypass the VPN using `bindProcessToNetwork`

- `setBlocking(boolean blocking)`: Controls file descriptor blocking mode

1.3 MTU Configuration

MTU configuration is critical for VPN performance. The default MTU of 1500 bytes may need adjustment based on the VPN protocol overhead:

```
"builder.setSession(getString(R.string.app_name)).addAddress('10.0.0.1', 24).addDnsServer('8.8.8.8').addRoute('0.0.0.0', 0).setMtu(1500)" [^27^]
```

For WireGuard, a typical MTU of 1420 is recommended (1500 - 80 bytes WireGuard overhead). For OpenVPN over UDP, 1400-1420 is common.

1.4 DNS Configuration

DNS leak prevention requires explicit DNS server configuration:

```
"You need to set your own DNS servers inside the VpnService.Builder on Android and the NEPacketTunnelNetworkSettings on iOS. Many devs miss this step and ship apps that leak DNS even when the tunnel is up." [^115^]
```

Android 10+ supports HTTP proxy via `VpnService.Builder.setHttpProxy()`:

```
"In Android 10, VPN apps can set an HTTP proxy for their VPN connection. To add an HTTP proxy, the VPN app must first configure a ProxyInfo instance with the host and port, then call VpnService.Builder.setHttpProxy()." [^165^]
```

1.5 Split Tunneling

Android provides two mutually exclusive approaches for split tunneling [^25^][^29^]:

Allowed Applications (Whitelist Mode):

```
builder.addAllowedApplication("com.example.app1")
builder.addAllowedApplication("com.example.app2")
// Only these apps use VPN; all others bypass
```

Disallowed Applications (Blacklist Mode):

```
builder.addDisallowedApplication("com.bank.app")
// This app bypasses VPN; all others use it
```

"A Builder may have only a set of allowed applications OR a set of disallowed ones, but not both. Calling this method after addDisallowedApplication has already been called, or vice versa, will throw an UnsupportedOperationException." [^25^]

For IP-based split tunneling (Android 33+), `builder.excludeRoute()` is available:

"For Android 33+, we used `builder.excludeRoute` to exclude the desired IPs from the VPN. For versions below Android 33, we relied on `builder.addRoute` and included the required IP addresses calculated using the WireGuard AllowedIPs Calculator." [^21^]

1.6 Always-On VPN and Lockdown Mode

Android supports always-on VPN starting from API 24 (Android 7.0):

"Always-on VPN, available since Android 7.0, starts the VPN service automatically on device boot and keeps it running. The connection persists across reboots and app updates without user interaction." [^26^]

Lockdown mode (API 29+/Android 10+) blocks all non-VPN traffic:

"When `lockdownEnabled` is true, all network traffic is blocked if the VPN is not connected. No traffic can leak to the open internet." [^26^]

Programmatic detection:

```
// Check if running as always-on VPN
val isAlwaysOn = isAlwaysOn() // VpnService method (API 29+)

// Check if lockdown mode is active
val isLockdown = isLockdownEnabled() // VpnService method (API 29+)
```

MDM/Enterprise configuration via
`DevicePolicyManager.setAlwaysOnVpnPackage()`:

"Lockdown mode ensures that all network traffic from Android endpoints passes through the Prisma Access Agent app, thereby enforcing security policies and preventing unauthorized access. When enabled, it blocks all network traffic on the endpoint until a connection to the Prisma Access Agent is established." [^160^]

1.7 Enterprise/MDM VPN Configuration

Android Enterprise supports various VPN types via MDM [^161^]:

- Cisco AnyConnect
- SonicWall Mobile Connect
- F5 Access
- Pulse Secure
- Microsoft Tunnel
- Palo Alto Networks

Always-On VPN requires Device Owner provisioning for enterprise scenarios [^162^]:

"Always On can be configured only for devices provisioned as Device Owner."

2. Android Background Execution

2.1 Foreground Service Requirement

Android VPN services must run as foreground services with persistent notifications:

"Your service needs to run as a foreground service with a notification. Background battery optimization can kill your service -- handle this with `PowerManager.WakeLock` carefully." [^115^]

```
class MyVpnService : VpnService() {
    override fun onStartCommand(intent: Intent?, flags: Int,
        startId: Int): Int {
        // Create notification channel (Android 26+)
        val channel = NotificationChannel("vpn", "VPN Service",
            NotificationManager.IMPORTANCE_LOW)
        getSystemService(NotificationManager::class.java).createNotificationChannel(channel)

        // Build persistent notification
        val notification = NotificationCompat.Builder(this, "vpn")
            .setContentTitle("VPN Connected")
            .setContentText("Securing your connection")
            .setSmallIcon(R.drawable.vpn_icon)
            .setOngoing(true)
            .build()

        // Start as foreground service
        startForeground(1, notification)

        return START_STICKY // Restart if killed
    }
}
```

```
}  
}
```

2.2 Doze Mode and App Standby

Doze mode (introduced in Android 6.0) affects VPN background operation:

"Doze mode and App Standby may temporarily suspend network and background execution for your app when it goes to background and the phone is idle." [^78^]

"An app that is whitelisted can use the network and hold partial wake locks during Doze and App Standby. However, other restrictions still apply to the whitelisted app, just as they do to other apps." [^87^]

2.3 Battery Optimization Whitelisting

Apps can request exemption from battery optimizations:

"Apps that meet acceptable use cases can call an intent with ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS to allow users to add the app directly to the exemption list without going to system settings." [^84^]

```
// Check if already whitelisted  
val powerManager = getSystemService(Context.POWER_SERVICE) as  
    PowerManager  
val isIgnoring =  
    powerManager.isIgnoringBatteryOptimizations(packageName)  
  
// Request whitelist (for non-Play Store apps)  
if (!isIgnoring) {  
    val intent =  
        Intent(Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS)  
    intent.data = Uri.parse("package:$packageName")  
    startActivity(intent)  
}
```

Important Play Store restriction:

"Please DO NOT use the above solution if you are distributing your app through Google Play, as it will most likely lead to app suspension." [^78^]

For Play Store apps, use ACTION_IGNORE_BATTERY_OPTIMIZATION_SETTINGS instead, which opens settings for manual user action.

2.4 WorkManager for Periodic Tasks

WorkManager can be used for periodic VPN health checks and reconnection attempts, though it's subject to Doze mode deferrals on API 23 and below.

2.5 Manufacturer-Specific Optimizations

"This is particularly important with manufacturers like OnePlus and Nokia (HMD Global) who customize Android to do aggressive battery optimization, keeping the device in Doze mode longer than intended." [^81^]

3. iOS VPN Development

3.1 Network Extension Framework

iOS VPN development uses the NetworkExtension framework, specifically NEPacketTunnelProvider for custom VPN protocols:

"The Network Extension framework is how an iOS app reaches into the network stack: building a VPN, a content filter, or a DNS proxy that can see and route the device's traffic." [^187^]

3.2 NEPacketTunnelProvider Implementation

The core of an iOS VPN is the NEPacketTunnelProvider subclass:

```
class PacketTunnelProvider: NEPacketTunnelProvider {
    override func startTunnel(
        options: [String: NSObject]?,
        completionHandler: @escaping (Error?) -> Void
    ) {
        // Read configuration
        guard let proto = protocolConfiguration as?
            NETunnelProviderProtocol else {
            completionHandler(NEPNError(.configurationInvalid))
            return
        }

        // Extract configuration from providerConfiguration
        let wgQuickConfig = proto.providerConfiguration?
            ["wgQuickConfig"] as? String

        // Setup tunnel network settings
```

```

let settings =
  NEPacketTunnelNetworkSettings(tunnelRemoteAddress:
    serverAddress)
  settings.ipv4Settings = NEIPv4Settings(addresses:
    ["10.0.0.2"], subnetMasks: ["255.255.255.0"])
  settings.dnsSettings = NEDNSSettings(servers: ["1.1.1.1",
    "8.8.8.8"])

  setTunnelNetworkSettings(settings) { error in
    completionHandler(error)
  }
}

override func stopTunnel(
  with reason: NEProviderStopReason,
  completionHandler: @escaping () -> Void
) {
  // Cleanup
  completionHandler()
}
}

```

From the WireGuard Flutter integration [^126^]:

```

final class PacketTunnelProvider: NEPacketTunnelProvider {
  private var adapter: WireGuardAdapter?

  override func startTunnel(options: [String : NSObject]?,
    completionHandler: @escaping (Error?) -> Void) {
    guard let proto = protocolConfiguration as?
      NETunnelProviderProtocol,
      let wgQuick = proto.providerConfiguration?
        ["wgQuickConfig"] as? String else {
      completionHandler(NSError(domain: "WG", code: -1))
      return
    }
    do {
      let cfg = try TunnelConfiguration(fromWgQuickConfig:
        wgQuick, called: nil)
      adapter = WireGuardAdapter(with: self) { _, msg in
        NSLog("[WireGuard] %@", msg)
      }
      adapter?.start(tunnelConfiguration: cfg) { err in
        completionHandler(err)
      }
    } catch { completionHandler(error) }
  }
}

```

3.3 NETunnelProviderManager

The main app uses NETunnelProviderManager to configure and control the VPN:


```
NETunnelProviderManager.loadAllFromPreferences { managers, error
    in
        // managers: [NETunnelProviderManager] - existing
        // configurations
}
}
```

Key operations:

- loadAllFromPreferences(): Load saved VPN configurations
- saveToPreferences(): Save VPN configuration to system settings
- connection.startVPNTunnel(): Start the VPN tunnel
- connection.stopVPNTunnel(): Stop the VPN tunnel

3.4 Network On-Demand Rules

VPN On-Demand automatically connects/disconnects based on network conditions:

```
let onDemandRule = NEOnDemandRuleConnect()
onDemandRule.interfaceTypeMatch = .any
manager.isOnDemandEnabled = true
manager.onDemandRules = [onDemandRule]
```

Wi-Fi specific on-demand [^64^]:

```
let onDemandRule = NEOnDemandRuleConnect()
onDemandRule.interfaceTypeMatch = .wiFi
onDemandRule.ssidMatch = ["OfficeNetwork"]
manager.onDemandRules = [onDemandRule]
manager.isOnDemandEnabled = true
```

"VPN On Demand is one of the options of NETunnelProviderManager that allows the system to automatically start or stop a VPN connection based on various criteria." [^131^]

3.5 Keychain Integration for Credentials

iOS VPN credentials should be stored securely in the Keychain:

```
// Store password reference in Keychain
let query: [String: Any] = [
    kSecClass as String: kSecClassGenericPassword,
    kSecAttrAccount as String: account,
    kSecReturnPersistentRef as String: true
]
var item: CTypeRef?
let status = SecItemCopyMatching(query as CFDictionary, &item)
```

"Store necessary connection info in App Group shared storage for recovery." [^64^]

3.6 Personal VPN vs Network Extensions

Apple provides two paths for VPN development [^64^]:

Personal VPN (NEVPNManager):

- Uses built-in IPsec or IKEv2 protocols
- Simpler implementation
- Consumer-facing VPN services
- System-wide traffic routing

Network Extensions (NEPacketTunnelProvider):

- Custom VPN protocols (WireGuard, OpenVPN, etc.)
- Content filtering capabilities
- Custom DNS solutions
- Low-level traffic inspection

"Choose Personal VPN when: Your primary goal is to provide a standard VPN connection using the widely supported IPsec or IKEv2 protocols. Choose Network Extensions when: You need to implement a custom VPN protocol that is not IPsec or IKEv2." [^64^]

4. iOS Constraints & Considerations

4.1 Memory Limits (~15MB)

The most critical constraint for iOS VPN Network Extensions is the strict memory limit:

"The library increasingly uses memory and breaks the limit of 15MB of NEPacketTunnelProvider limit. This causes the extension process to be killed by the ios. Thread 14: EXC_RESOURCE (RESOURCE_TYPE_MEMORY: high watermark memory limit exceeded) (limit=15 MB)" [^83^]

"Apple technical support has confirmed that there is a 5-6MB memory limit for network extensions, raised to 15MB in iOS 10 Beta 2." [^90^]

This 15MB limit encompasses ALL memory used by the extension process including:

- Protocol implementation
- Encryption/decryption buffers
- Connection state tracking
- Logging

Mitigation strategies:

- Use native Rust/Go implementations with minimal Swift wrapper overhead
- Limit buffer sizes and connection tracking state
- Avoid excessive logging in production
- Use shared App Groups for state that doesn't need to be in the extension
- Implement zero-copy strategies where possible

4.2 Background Execution Rules

iOS Network Extensions run as separate system processes, independent of the host app:

"The answer is a resounding yes, and this capability is a cornerstone of the Network Extension framework's design. Apple has engineered this framework to allow VPN connections to operate at a system level, independent of the lifecycle of the application that configured them." [^64^]

Key behaviors:

- VPN persists even when main app is terminated
- Extension is managed by the OS kernel
- Sleep/wake events must be handled to restore state
- App Groups shared storage for recovery data

4.3 App Store Review Considerations

Apple scrutinizes VPN apps heavily:

"A traffic-handling component holds high trust: it can inspect or route traffic. Using it is a serious responsibility: scrutinized in App Store review." [^187^]

Key requirements [^186^]:

- **Privacy Policy:** Must accurately describe data collection, use, and retention
- **Network Extension Entitlement:** Requires Apple-granted capability
- **Battery Impact:** Apps that drain battery may be rejected; WireGuard typically passes
- **ATT (App Tracking Transparency):** Applies if collecting tracking data
- **User Authorization:** User must explicitly approve VPN configuration

"Network Extension API: VPN apps use this to create the tunnel. Apple reviews how the extension behaves -- background usage, battery impact, and whether it complies with their network extension guidelines." [^186^]

Common rejection reasons:

- Incomplete or vague privacy policy
- Misleading claims (e.g., "military-grade" without basis)
- Metadata that doesn't match app functionality
- Network extension behavior that violates guidelines

4.4 Entitlements

Required entitlements for packet tunnel VPN:

```
<key>com.apple.developer.networking.networkextension</key>
<array>
  <string>packet-tunnel-provider</string>
</array>
```

"You don't need a Personal VPN entitlement which allows apps to create and control a custom system VPN configuration using NEVPNManager. The Packet Tunnel Provider entitlements are classified as enterprise VPNs and only require Network Extension entitlement." [^131^]

5. HarmonyOS VPN Development

5.1 VpnExtensionAbility API

HarmonyOS NEXT introduces VpnExtensionAbility for third-party VPN development [^77^]:

```
import { VpnExtensionAbility } from '@kit.NetworkKit';
import { Want } from '@kit.AbilityKit';

class MyVpnExtAbility extends VpnExtensionAbility {
  onCreate(want: Want) {
    console.info('MyVpnExtAbility onCreate');
    // Initialize VPN resources
  }

  onDestroy() {
    console.info('MyVpnExtAbility onDestroy');
    // Cleanup VPN resources
  }
}
```

Key HarmonyOS VPN APIs:

- `VpnExtensionAbility`: Lifecycle callbacks for VPN creation/destruction
- `VpnExtensionContext`: Context for the VPN extension
- `vpnExtension.createVpnConnection(context)`: Create VPN connection
- `vpnConnection.create(vpnConfig)`: Establish VPN with configuration

5.2 Native N-API Bridge

HarmonyOS uses Node-API (N-API) to bridge ArkTS with native C/C++ code:

"HarmonyOSNext Node-API bridges ArkTS/JS and native C/C++ modules with a stable, cross-platform API. Developed based on Node.js 12.x LTS Node-API." [^130^]

Key N-API functions for native interop:

- `napi_wrap`: Binds ArkTS object to native C++ instance
- `napi_unwrap`: Retrieves native C++ object from wrapped ArkTS counterpart
- `napi_create_function`: Exposes C++ functions to ArkTS

5.3 Native C++ Integration

From the Hey VPN project [^22^], the HarmonyOS VPN native path:

User connects

- > `vpnExtension.startVpnExtensionAbility(...)`
- > `HeyVpnAbility.onCreate(want)`
- > `vpnExtension.createVpnConnection(context)`
- > `vpnConnection.create(vpnConfig)`
- > TUN fd
- > `libheyvpn.so dlopen(libxray.so)`
- > `CGoSetTunFd(tunFd)`
- > `CGoRunXrayFromJSON(config)`
- > Xray native TUN inbound reads the Harmony VPN fd
- > Xray outbound

5.4 DevEco Studio Development

HarmonyOS development uses DevEco Studio:

"DevEco Studio: supports app and service development of the HarmonyOS NEXT Developer Preview version. It offers new features like reconstruction of codes, visual analysis of

construction, multiple in-depth tuning, app physical examination, and ARM version of the local simulator."
[^30^]

Key requirements:

- DevEco Studio / HarmonyOS SDK 6.1.1, API 24
- Stage model (not FA model)
- Native C++ projects use CMake for build configuration

5.5 HarmonyOS NEXT Specifics

Key differences from Android/iOS:

- **No global app enumeration:** HarmonyOS NEXT restricts listing all installed apps, affecting per-app proxy features
[^22^]
- **VPN Authorization:** Some emulator/system images do not include the VPN authorization component [^22^]
- **Stage Model:** Apps use EntryAbility, VPN Extension Ability, and backup ability architecture
- **Native Libraries:** Uses .so files packaged in entry/src/main/cpp/prebuilt/arm64-v8a/

6. HarmonyOS Cross-Platform Strategy

6.1 Framework Comparison

Solution	Owner	Language	Maturity	Code Reuse	Performance	Misc
Flutter	Google	Dart	Most mature for HarmonyOS	~85-90%	High (Skia/Impeller)	A c
React Native	Meta	JS/TS	Community-maintained OHOS	~75-80%	Medium (Native components)	M
uni-app x	DCloud	Vue/UTS	Thousands of plugins	~90%+	Fast (Native rendering)	D o
ArkUI-X	Huawei	ArkTS	Official, cross-device	N/A (HarmonyOS-first)	Highest	H
Taro	JD	React/Vue	C-API version available	High	Good	J
Kuikly	Tencent	Kotlin		High	Very fast (122ms FCP)	T

Solution	Owner	Language	Maturity	Code Reuse	Performance	M
			Kotlin/ Native solution			

[^3^][^89^]

6.2 Recommended Framework Choice for VPN Apps

For VPN apps targeting HarmonyOS, the analysis suggests:

1. Flutter (Recommended for cross-platform):

"Flutter, RN, and uni are relatively mature cross-platform solutions for HarmonyOS, and many large enterprises and central enterprises have used them in their APPs."

[^3^]

- Released version 3.22.0-ohos, deeply integrated with HarmonyOS NEXT API16
- Supports dual rendering engines: Skia/Impeller
- Best for teams with existing Flutter expertise

2. ArkUI-X (Recommended for HarmonyOS-first):

- Native HarmonyOS integration with ArkTS
- Best performance on HarmonyOS devices
- ArkUI-X extends to Android/iOS

3. uni-app x (Recommended for Vue teams):

"Fast startup, low memory usage, native components + native rendering performance close to native, logic layer and view layer share native process for rapid response."

[^3^]

- Thousands of plugins for HarmonyOS NEXT
- Well-known e-commerce app cases

6.3 Code Reuse Strategy

For maximum code reuse across Android, iOS, and HarmonyOS:

```
Shared Rust Core (VPN protocol, encryption, state machine)
|-- JNI -- Android (Kotlin) -- Flutter/React Native UI
|-- FFI -- iOS (Swift) -- Flutter/React Native UI
|-- N-API -- HarmonyOS (ArkTS) -- ArkUI/Flutter UI
```

6.4 Huawei AppGallery Requirements

- Apps must be signed with Huawei certificates
 - HAP (HarmonyOS Ability Package) format
 - Requires Huawei Developer account
 - Security review for VPN apps (similar to Apple/Google scrutiny)
-

7. Mobile Rust Core Integration

7.1 Architecture Overview

Using Rust as a shared core for mobile VPN apps is a proven pattern. Mullvad VPN exemplifies this architecture:

"The client consists of two main parts - the daemon and the GUI, but there's also a CLI. The daemon is the process that's responsible for upholding the security guarantees of the client, it consists of an actor system so that it can drive many processes asynchronously." [^23^]

Rust Daemon (VPN core)

```
|-- JNI -- Android Frontend (Kotlin/Java)
|-- gRPC -- Desktop Frontend
|-- FFI/iOS -- iOS Frontend (Swift)
```

7.2 Android: JNI/NDK Bindings

Using cargo-ndk for cross-compilation:

Install cargo-ndk

```
cargo install cargo-ndk
```

Add Android targets

```
rustup target add \
    aarch64-linux-android \
    armv7-linux-androideabi \
    i686-linux-android \
    x86_64-linux-android
```

Build shared libraries

```
cargo ndk -o ./app/src/main/jniLibs \
    --manifest-path ./Cargo.toml \
    -t armeabi-v7a \
    -t arm64-v8a \
    -t x86 \
    -t x86_64 \
    build --release
```


[^79^]

JNI integration:

```
// Kotlin calling Rust through JNI
external fun nativeStartTunnel(config: String): Int
external fun nativeStopTunnel(): Int
external fun nativeGetStats(): TunnelStats
```

7.3 iOS: FFI/C-Bindings

Using cargo-lipo for universal libraries:

```
# Add iOS targets
rustup target add aarch64-apple-ios
rustup target add aarch64-apple-ios-sim

# Build for device and simulator
cargo build --target aarch64-apple-ios --release
cargo build --target aarch64-apple-ios-sim --release

# Create universal binary with lipo
lipo -create \
    target/aarch64-apple-ios/release/libmyvpn.a \
    target/aarch64-apple-ios-sim/release/libmyvpn.a \
    -output libmyvpn.a
```

[^85^]

7.4 UniFFI for Cross-Platform Bindings

UniFFI generates bindings for both Android (Kotlin) and iOS (Swift):

```
// Rust library with UniFFI
uniffi::setup_scaffolding!();

#[uniffi::export]
pub fn start_tunnel(config: String) -> Result<TunnelState,
    VpnError> {
    // Implementation
}

#[uniffi::export]
pub fn stop_tunnel() -> Result<(), VpnError> {
    // Implementation
}
```

Generating Swift bindings:

```
cargo run --bin uniffi-bindgen generate \
    --library ./target/debug/libswitzerland.dylib \
```

```
--language swift \  
--out-dir ./bindings
```

[^82^]

Generated Swift code:

```
// Generated by UniFFI  
public func neutralGreeting() -> String {  
    return try! FfiConverterString.lift(try! rustCall() {  
        uniffi_switzerland_fn_func_neutral_greeting($0)  
    })  
}
```

[^82^]

7.5 Mullvad's GotaTun: Rust WireGuard in Production

Mullvad recently replaced wireguard-go with GotaTun (Rust implementation):

"GotaTun is a WireGuard implementation written in Rust aimed at being fast, efficient and reliable. GotaTun is a fork of the BoringTun project from Cloudflare." [^62^]

Results after Android rollout:

- **User-perceived crash rate: 0.40% -> 0.01%**
- **Not a single crash** has stemmed from GotaTun
- **Better speeds and lower battery usage** reported by users

"Since rolling out GotaTun on Android with version 2025.10 in the end of November we've seen a big drop in the metric user-perceived crash rate, from 0.40% to 0.01%." [^62^]

Why Rust over Go for mobile:

"Another challenge we have faced is interoperating Rust and Go. Since Go is a managed language with its own separate runtime, how it executes is opaque to the Rust code. If wireguard-go were to hang or crash, recovering stacktraces is not always possible which makes debugging the code cumbersome." [^62^]

7.6 React Native UniFFI Integration

For React Native apps, uniffi-bindgen-react-native provides TurboModule integration:

"uniffi-bindgen-react-native is configured to generate Swift bindings but fails to do so silently, leaving only C++/Objective-C++ files that don't properly integrate with React Native's New Architecture TurboModule system." [^28^]

Key considerations:

- New Architecture (TurboModules) required
 - Swift bindings may need manual integration
 - Android integration via JNI is more mature
-

8. Mobile UI Patterns for VPN Apps

8.1 Common VPN Mobile UI Patterns

Standard VPN app UI components:

- **Connection Button:** Large, prominent connect/disconnect toggle
- **Server Selection List:** Searchable list with latency indicators
- **Connection Stats:** Real-time upload/download speeds, data usage
- **Settings:** Protocol selection, kill switch, split tunneling
- **Quick Connect:** Connect to optimal server with one tap

8.2 Flutter VPN UI Architecture

Example from wireguard_flutter_plus [^119^]:

```
class _MyAppState extends State<MyApp> {
  final wireguard = WireGuardFlutter.instance;
  String vpnState = VpnEngine.vpnDisconnected;
  StreamSubscription? _vpnStatusSubscription;
  StreamSubscription? _vpnTraffic;

  // Traffic stats
  String downloadCount = "0.0 B/s";
  String uploadCount = "0.0 B/s";
  String totalDownload = "0 B";
  String totalUpload = "0 B";
  String duration = "00:00:00";

  @override
  void initState() {
    super.initState();
    _vpnTraffic = wireguard.trafficSnapshot.listen((data) {
      setState(() {
        downloadCount =
          formatSpeed(data["downloadSpeed"]);
      });
    });
  }
}
```

```

        uploadCount = formatSpeed(data["uploadSpeed"]);
        totalDownload =
        formatBytes(data["totalDownload"]);
    });
});

_vpnStatusSubscription =
wireguard.vpnStageSnapshot.listen((event) {
    setState(() { vpnState = event.name; });
});
}
}

```

8.3 Android Quick Settings Tile

Android supports custom Quick Settings tiles for VPN toggle:

Requirements from QuickTile Settings [^122^]:

```

<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" /
>

```

Implementation requires:

1. Extend TileService class
2. Register in AndroidManifest.xml with BIND_QUICK_SETTINGS_TILE action
3. Handle click events to toggle VPN connection
4. Update tile state (active/inactive) based on VPN status

8.4 iOS Control Center Integration

iOS VPN apps appear in Settings > VPN and can be toggled from Control Center when added. The system handles the UI; the app provides:

- VPN configuration via NETunnelProviderManager
- Connection state via NEVPNStatusDidChange notifications
- On-demand rules for automatic connection

8.5 Persistent Notifications

Android VPN apps must show persistent notifications:

```

val notification = NotificationCompat.Builder(context,
    VPN_CHANNEL_ID)
    .setContentTitle("VPN Connected")
    .setContentText("Server: $serverLocation | Protocol:
    $protocol")

```

```
.setSmallIcon(R.drawable.ic_vpn_connected)
.setOngoing(true) // Cannot be dismissed
.setPriority(NotificationCompat.PRIORITY_LOW)
.addAction(R.drawable.disconnect, "Disconnect",
    disconnectPendingIntent)
.build()
```

9. Push Notifications for VPN Apps

9.1 Connection Status Notifications

VPN apps use notifications for:

- Connection established/terminated events
- Reconnection attempts
- Server maintenance alerts
- Subscription expiry warnings

9.2 FCM (Firebase Cloud Messaging) for Android

"FCM can deliver a message to an Android device instantly (push) without the app itself having to be running or the device having to poll -- the 'always-on' connection is the conduit." [^164 ^]

Key characteristics:

- Single persistent TCP connection shared across all FCM apps
- Negligible battery impact when idle
- High-priority messages can wake device in Doze mode

9.3 APNs (Apple Push Notification Service) for iOS

iOS uses APNs for push delivery:

- Device maintains persistent connection to APNs
- System (not app) receives notification
- App can be woken to process notification

9.4 Keep-Alive via On-Demand Rules (iOS)

Instead of push notifications, iOS VPN apps typically use On-Demand rules for reconnection:

"Disabling keep-alive will not result in any data leak outside of the VPN, as the on-demand rules will automatically re-establish the VPN tunnel before any network traffic starts on the device." [^124^]

Trade-offs:

- **Keep-alive ON:** Maintains persistent tunnel, faster response but more battery drain
 - **Keep-alive OFF:** Better battery life, but occasional delays for reconnection
-

10. Biometric Authentication

10.1 Android BiometricPrompt

Android provides BiometricPrompt for fingerprint/face authentication:

```
val biometricPrompt = BiometricPrompt(activity, executor,
    object : BiometricPrompt.AuthenticationCallback() {
        override fun onAuthenticationSucceeded(
            result: AuthenticationResult
        ) {
            // Unlock VPN credentials
        }
    })

val promptInfo = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Authenticate to Connect VPN")
    .setSubtitle("Verify your identity")
    .setAllowedAuthenticators(
        BIOMETRIC_STRONG or DEVICE_CREDENTIAL
    )
    .build()
```

```
biometricPrompt.authenticate(promptInfo)
```

10.2 iOS Face ID / Touch ID

iOS uses LocalAuthentication framework and Keychain SecAccessControl:

```
let context = LAContext()
context.evaluatePolicy(.deviceOwnerAuthenticationWithBiometrics,
    localizedReason: "Authenticate to access VPN") { success,
    error in
    if success {
        // Access Keychain-protected credentials
    }
}
```

```
}  
}
```

10.3 Secure Credential Storage with Biometrics

Android: Use `CryptoObject` to bind biometric authentication to `KeyStore` keys:

```
"When we call the authenticate function with a non-null  
cipher value and authenticationValidityDurationSeconds ==  
-1, then the BiometricPrompt is called with the CryptoObject  
wrapping our cipher." [^192^]
```

```
// Cipher protected by biometric authentication  
val cipher = cipherForMode() // Requires biometric auth  
prompt.authenticate(promptBuilder.build(),  
    BiometricPrompt.CryptoObject(cipher))
```

iOS: Use `SecAccessControl` with biometric constraints:

```
"For iOS implementation, the security access control is  
defined with kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly  
and biometryCurrentSet" [^192^]
```

10.4 Flutter Biometric Storage

The `biometric_storage` plugin provides cross-platform biometric-protected storage:

```
final authStorage = await BiometricStorage().getStorage(  
    'authenticated_storage',  
    options: StorageFileInitOptions(  
        authenticationValidityDurationSeconds: -1,  
        authenticationRequired: true,  
        androidBiometricOnly: true,  
    ),  
);  
  
// Write with biometric protection  
await authStorage?.write(secretData);  
  
// Read with biometric authentication  
final data = await authStorage?.read(); // Triggers biometric  
prompt
```

```
[^192^]
```

11. Mobile Performance Optimization

11.1 Battery Impact by Protocol

Protocol	Battery Impact	Idle Behavior	Reconnection
WireGuard	+1-3%/hr	Silent when sleeping	Instant (<1 sec)
IKEv2	+2-5%/hr	Minimal keepalives	Fast (1-2 sec)
OpenVPN UDP	+5-8%/hr	Periodic keepalives	Slow (5-15 sec)
OpenVPN TCP	+8-12%/hr	Frequent keepalives	Slowest (10-20 sec)

[^123^]

"WireGuard is also more efficient when your phone is just sitting in your pocket. Older protocols like OpenVPN are more chatty and send keepalive packets to hold the connection open, which can keep waking your phone's cellular radio." [^123^]

11.2 Battery Optimization Strategies

1. **Use WireGuard:** Most battery-efficient protocol
2. **Smart Keep-Alive:** Disable keep-alive during sleep; rely on on-demand rules (iOS)
3. **Wi-Fi Preference:** VPN on Wi-Fi drains significantly less than on cellular
4. **Hardware Acceleration:** Modern phones (2022+) have AES/ChaCha20 hardware acceleration
5. **Idle Connection Handling:** Reduce heartbeat frequency when idle

"Disabling keep-alive will improve battery consumption when the device is in sleep mode, but the drawback is you might experience occasional slowdowns due to wake-up reconnections." [^124^]

11.3 Connection Idle Handling

Best practices for idle connection management:

- Implement adaptive keepalive intervals (longer when idle)
- Use WireGuard's silent-when-idle design
- Configure PersistentKeepalive only when behind NAT

- Implement smart reconnection on network changes

11.4 Smart Reconnection Strategies

```
// Android: Monitor network changes
val connectivityManager =
    getSystemService(ConnectivityManager::class.java)
connectivityManager.registerDefaultNetworkCallback(object :
    NetworkCallback() {
        override fun onAvailable(network: Network) {
            // Rebind tunnel to new network
            rebindTunnel(network)
        }

        override fun onLost(network: Network) {
            // Tunnel will use default network
        }
    })
```

From React Native WireGuard implementation [^190^]:

```
private val netCb = object :
    ConnectivityManager.NetworkCallback() {
        override fun onAvailable(n: Network) {
            Log.d(TAG, "Network available -> rebind")
            rebind(n)
        }
        override fun onLost(n: Network) {
            Log.d(TAG, "Network lost -> rebind to default")
            rebind(null)
        }
    }
}
```

12. Real-World Mobile VPN App Architectures

12.1 Mullvad VPN

Architecture: Rust daemon + platform-specific frontend

```
Rust Daemon (talpid-core + mullvad-daemon)
|-- JNI --> Android (Kotlin UI)
|-- Unix Domain Socket --> Desktop
|-- FFI/NetworkExtension --> iOS (Swift UI)
```

[^23^]

Key technical decisions:

- **Daemon pattern:** Unprivileged frontend + privileged system service
- **WireGuard protocol:** Using GotaTun (Rust) on Android, wireguard-go on iOS
- **Frontend communication:** gRPC (desktop), JNI (Android)
- **Offline detection:** ConnectivityManager on Android, NWPathMonitor on iOS

Android implementation details:

"The frontend communicates with the daemon via the management interface and these are serviced asynchronously via the daemon. Frontends can also subscribe to messages from the daemon, to receive updates about the tunnel state, new settings, new relay lists, version information and device events." [^23^]

"To detect connectivity on Android, the app relies on ConnectivityManager by listening for changes to the availability of non-VPN networks that provide internet connectivity." [^23^]

iOS implementation details:

"The iOS app uses WireGuard kit's offline detection, which in turn uses NWPathMonitor to listen for changes to the route table and assumes connectivity if a default route exists." [^23^]

Protocols used:

- WireGuard (primary): WireGuardNT (Windows), GotaTun (Android), wireguard-go (iOS/macOS)
- OpenVPN: Recently removed from desktop, still on some mobile
- TLS 1.3 with Rustls for API communication
- Certificate pinning to prevent MitM

[^20^]

12.2 ExpressVPN / NordVPN

These commercial VPNs typically use:

- **Native development:** Swift (iOS), Kotlin/Java (Android)
- **WireGuard or custom protocols:** NordVPN uses NordLynx (WireGuard-based)
- **Custom UI:** Native platform UI components
- **Lightway:** ExpressVPN's custom protocol optimized for mobile

12.3 Flutter-Based VPN Apps

Emerging pattern for cross-platform VPN apps:

"To develop a Flutter VPN mobile app, use Flutter for the UI, server selection, login, subscriptions, and app state, while handling the actual VPN tunnel through native Android VpnService and iOS NetworkExtension using platform channels." [^115^]

Architecture:

Flutter UI Layer (Dart)

- |-- MethodChannel --> Android VpnService (Kotlin)
- |-- MethodChannel --> iOS NetworkExtension (Swift)
- |-- Dart FFI --> Shared Rust Core (optional)

Development timeline estimate:

"Realistically: 3-4 months for a small team with mobile experience. 6+ months if the team is learning the VPN stack for the first time. Most of the time goes to native code, edge cases, and App Store approvals -- not the Flutter UI." [^115^]

12.4 React Native VPN Apps

React Native VPN modules exist but require significant native development:

"React-Native does not have a networking extension library, so I had to write an extension that supports iOS." [^191^]

Available packages:

- react-native-wireguard-vpn-connect: WireGuard for iOS/Android [^188^]
- react-native-wireguard-vpn: WireGuard with Expo config plugin [^189^]

13. Platform Comparison Matrix

13.1 VPN Feature Comparison

Feature	Android	iOS
VPN API	VpnService Any (custom)	NEPacketTunnel

Feature	Android	iOS
Protocol Support		Any via NetworkExtension
Split Tunneling	addAllowedApplication() / addDisallowedApplication()	Per-app via router rules
Always-On VPN	Yes (API 24+)	Via On-Demand VPN
Lockdown Mode	Yes (API 29+)	includeAllNetworks
Foreground Service	Required	N/A (extension runs independently)
Background Persistence	START_STICKY + foreground	System-managed extension
Memory Limit	~200MB+ (device dependent)	~15MB (NetworkExtension)
Quick Toggle	Quick Settings Tile	Control Center (Settings)
Battery Optimization	REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	N/A (system manages)
Biometric Auth	BiometricPrompt	LocalAuthentication / Keychain
Credential Storage	Android Keystore	iOS Keychain
Push Notifications	FCM	APNs
Rust Integration	JNI/NDK	FFI/C-bindings
Code Signing	APK/AAB signing	Apple certificate
Simulator Support	Emulator (limited VPN)	Physical device

13.2 Development Complexity Comparison

Aspect	Android	iOS	HarmonyOS
Setup Complexity	Medium	High (entitlements, provisioning)	Medium (DevEco Studio)
Native Code Required	Kotlin/Java	Swift/Objective-C	ArkTS + C++
Debugging		Xcode + device required	DevEco Studio

Aspect	Android	iOS	HarmonyOS
	Android Studio + logcat		
App Store Review	Moderate	Strict (VPN scrutiny)	Moderate
Documentation	Excellent	Good	Growing
Community	Very large	Large	Growing

14. Mobile-Specific Danger Zones

14.1 Critical Issues

#	Danger Zone	Impact	Mitigation
1	iOS 15MB Memory Limit	Extension killed, VPN disconnects	Minimal Swift/ObjC code; delegate to Rust/C; zero-copy buffers
2	Android Doze Mode	Background service killed	Foreground service + notification; battery whitelist request
3	Manufacturer Battery Optimizations	(OnePlus, Samsung, Xiaomi) Aggressive app killing	Guide users to disable optimizations; use foreground service
4	iOS Network Extension Entitlement	Cannot develop without Apple approval	Apply for entitlement early; use paid developer account
5	DNS Leaks	DNS traffic bypasses VPN	Explicit DNS configuration on both platforms
6	Split Tunneling Misconfiguration	Traffic leaks outside VPN	Thorough testing with multiple route configurations
7	Credential Storage	Private keys exposed	Android Keystore / iOS Keychain with biometric protection
8	Protocol Battery Drain	Excessive battery usage	

#	Danger Zone	Impact	Mitigation
9	App Store VPN Rejection	App rejected or removed	Use WireGuard; optimize keepalive intervals Clear privacy policy; no misleading claims; minimal analytics
10	Rust FFI Complexity	Memory safety issues at boundaries	Use UniFFI; careful memory management; extensive testing
11	HarmonyOS App Enumeration	Cannot list all apps for split tunneling	Use preset app list; manual package entry
12	Network Change Handling	Tunnel drops on WiFi/cellular switch	Implement network callback rebind logic

14.2 Platform-Specific Gotchas

Android:

- BIND_VPN_SERVICE permission requires system dialog (cannot bypass)
- Always-on VPN conflicts with other VPN apps
- 16KB page sizes on Android 15+ require library recompilation
- Work profile / personal profile VPN separation on enterprise devices

iOS:

- Network Extension requires **physical device** for testing (simulator not supported)
- Memory limit is **hard** -- no way to increase it
- Extension bundle ID must be prefixed with app bundle ID
- NSExtensionPrincipalClass in Info.plist must match class name
- loadFromPreferences bug requires double-call before starting tunnel [^131^]

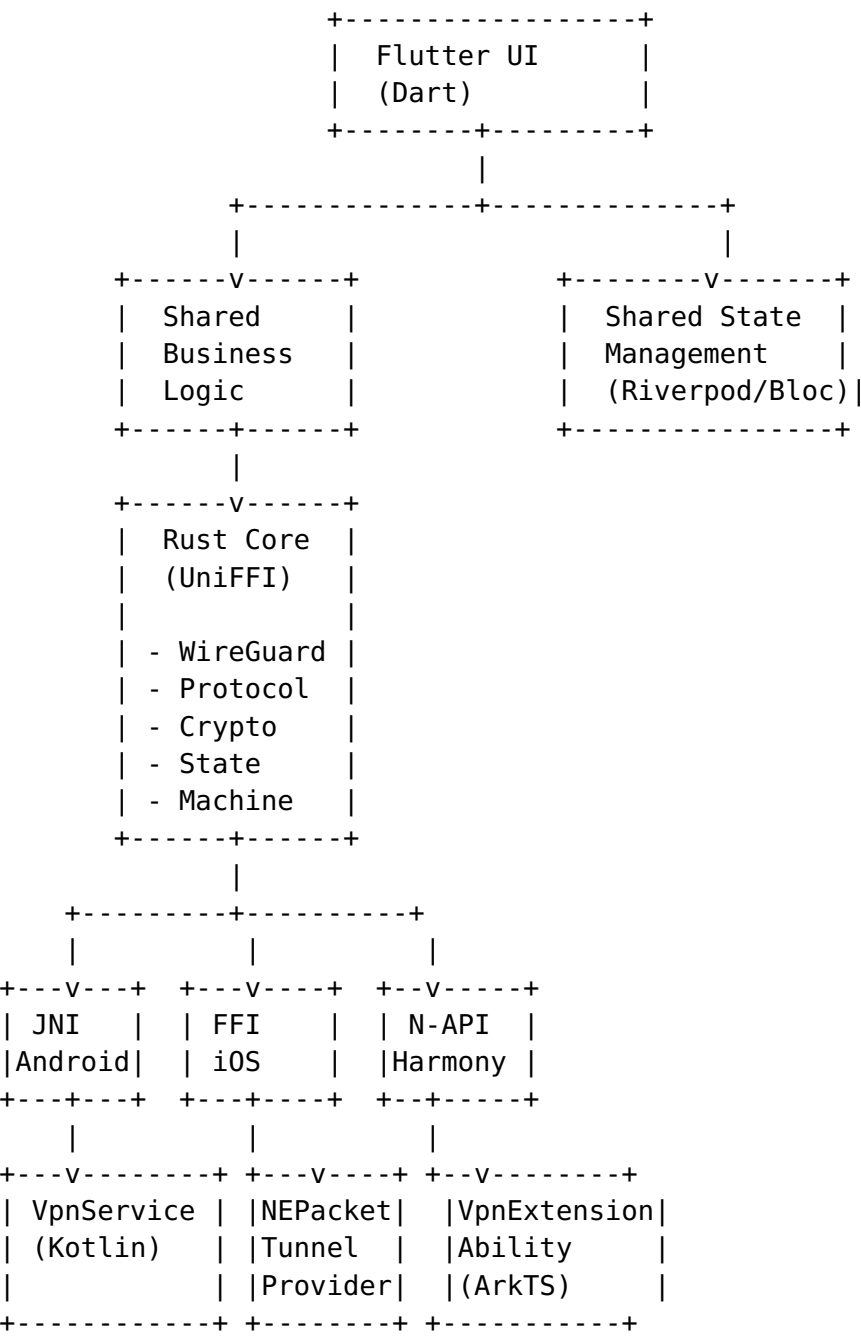
HarmonyOS:

- VPN authorization component may not be present on emulators
- Cannot enumerate all installed apps (privacy restriction)
- N-API bridge has learning curve
- Smaller ecosystem for third-party libraries

15. Recommended Architecture

15.1 Optimal Cross-Platform VPN Architecture

Based on research findings, the recommended architecture for a cross-platform VPN app:



15.2 Technology Stack Recommendations

Component	Recommendation	Rationale
UI Framework	Flutter	

Component	Recommendation	Rationale
VPN Core	Rust (GotaTun/Custom)	85-90% code reuse, mature ecosystem, good native bridge Memory safety, performance, unified codebase
Android Native	Kotlin + JNI	Platform idiomatic, good Rust interop
iOS Native	Swift + FFI	Minimal wrapper due to 15MB limit
HarmonyOS Native	ArkTS + N-API	Official language, native C++ bridge
Bindings	UniFFI	Automated Kotlin/Swift generation
State Management	Riverpod (Flutter)	Reactive, testable
Protocol	WireGuard	Best battery efficiency, modern crypto
Credentials	Platform Keystore	OS-level security
Auth	BiometricPrompt (Android) / LocalAuthentication (iOS)	Standard platform APIs

15.3 Implementation Priority

1. **Phase 1:** Rust core with WireGuard implementation + UniFFI bindings
2. **Phase 2:** Android VpnService integration with Kotlin JNI
3. **Phase 3:** iOS NetworkExtension with Swift (watch memory!)
4. **Phase 4:** Flutter UI layer with platform channels
5. **Phase 5:** HarmonyOS VpnExtensionAbility with N-API bridge
6. **Phase 6:** Biometric auth, push notifications, quick settings tiles
7. **Phase 7:** Battery optimization, enterprise features (always-on, lockdown)

15.4 Key Success Factors

1. **Rust core must be lean** -- iOS memory limit is the hardest constraint
2. **Native code must handle network changes gracefully** -- rebind on every network transition
3. **DNS configuration is critical** -- never rely on system DNS when VPN is active

4. **Battery optimization is a feature** -- users will uninstall battery-draining VPNs
 5. **App Store compliance from day one** -- privacy policy, minimal permissions, no misleading claims
 6. **Test on real devices** -- emulators don't reflect real-world networking behavior
 7. **Always-on VPN support** -- enterprise users expect this; consumer users benefit from it
-

Sources and References

- [^21^] Stack Overflow - Android VPN Split Tunneling: <https://stackoverflow.com/questions/79207428/>
- [^22^] Hey VPN - HarmonyOS VPN Client (GitHub): <https://github.com/popsiclelmlm/Hey>
- [^23^] Mullvad VPN Architecture (GitHub): <https://github.com/mullvad/mullvadvpn-app/blob/main/docs/architecture.md>
- [^25^] Android VpnService.java Source: <https://android.googlesource.com/platform/frameworks/base/+refs/heads/main/core/java/android/net/VpnService.java>
- [^26^] Jason Bayton - Android Enterprise VPN: <https://bayton.org/android/android-enterprise-faq/global-vpn-support/>
- [^27^] Medium - Android VPN Service Guide: <https://medium.com/@satish.nada98/complete-guide-to-implementing-a-vpn-service-in-android>
- [^28^] UniFFI React Native Integration Issue: <https://github.com/jhugman/uniffi-bindgen-react-native/issues/278>
- [^29^] Android Developer - VPN Guide: <https://developer.android.com/develop/connectivity/vpn>
- [^30^] Huawei Central - HarmonyOS NEXT: <https://www.huaweicentral.com/harmonyos-next-developer-preview-official-website-highlights-key-features/>
- [^3^] DEV Community - HarmonyOS Cross-Platform Solutions: <https://dev.to/georgegcs/harmonyos-5-detailed-explanation-of-harmonyos-cross-platform-development-solutions-part-1-30pn>
- [^62^] Mullvad GotaTun Announcement: <https://mullvad.net/en/blog/announcing-gotatun-the-future-of-wireguard-at-mullvad-vpn>
- [^64^] iOS Network Extensions Guide: <https://antongubarenko.substack.com/p/ios-personal-vpn-and-network-extensions>
- [^77^] HarmonyOS VpnExtensionAbility API: <https://developer.huawei.com/consumer/en/doc/harmonyos-references/js-apis-vpnextensionability>
- [^78^] Pushy - Android Doze Mode: <https://support.pushy.me/hc/en-us/articles/360043423332>

- [^79^] UniFFI Android Binding Guide: <https://jans.io/docs/v1.7.0/cedarling/uniffi/cedarling-android/>
- [^81^] Sentiance - Android Battery Optimization: <https://docs.sentiance.com/important-topics/sdk/appendix/android/android-battery-optimization>
- [^82^] Multiplatform with Rust on iOS: <https://mobilesystemdesign.substack.com/p/multiplatform-with-rust-on-ios-2c4>
- [^83^] hev-socks5-tunnel iOS Memory Issue: <https://github.com/heihher/hev-socks5-tunnel/issues/109>
- [^84^] Android Doze and App Standby: <https://developer.android.com/training/monitoring-device-state/doze-standby>
- [^85^] Cross-Platform Rust for Web, Android, iOS: <https://artificialworlds.net/blog/2022/07/06/building-cross-platform-rust-for-web-android-and-ios-a-minimal-example/>
- [^87^] Stack Overflow - Doze Mode Foreground Services: <https://stackoverflow.com/questions/56866806/doze-mode-do-foreground-services-continue-to-run>
- [^89^] HarmonyOS Developer - Cross-Platform Frameworks: <https://developer.harmonyos.cool/docs/resources/cross-platform/overview>
- [^90^] Open Radar - iOS Network Extension Memory Limit: <http://www.openradar.appspot.com/27660401>
- [^115^] Flutter VPN App Development: <https://appilian.com/flutter-vpn-mobile-app-development/>
- [^119^] wireguard_flutter_plus Example: https://pub.dev/packages/wireguard_flutter_plus/example
- [^122^] QuickTile Settings - F-Droid: <https://f-droid.org/en/packages/com.rbn.qtsettings/>
- [^123^] Windscribe - VPN Battery Usage: <https://windscribe.com/blog/does-vpn-drain-battery/>
- [^124^] IVPN - Battery Drain: <https://www.ivpn.net/knowledgebase/troubleshooting/the-battery-on-my-phone-drains-too-fast-while-using-ivpn-why/>
- [^126^] Flutter + WireGuard VPN: <https://medium.com/@mdazadhossain95/flutter-wireguard-vpn-one-codebase-android-and-ios-dedb9d4286ec>
- [^129^] Kean.blog - Packet Tunnel Provider: <https://kean.blog/post/packet-tunnel-provider>
- [^130^] Node-API on HarmonyOS: <https://medium.com/huawei-developers/node-api-part-1-introduction-to-node-api-wrapping-native-c-objects-in-arkts-on-0a743f5cc175>
- [^131^] VPN Configuration Manager: <https://github.com/kean/articles/blob/master/2020-05-24-vpn-configuration-manager.markdown>
- [^160^] Palo Alto Networks - Lockdown Mode: <https://docs.paloaltonetworks.com/>
- [^161^] Microsoft Intune - Android VPN: <https://learn.microsoft.com/en-us/intune/device-configuration/templates/ref-vpn-settings-android-enterprise>

- [^164^] Push Notification Internals: <https://blog.clix.so/how-push-notification-delivery-works-internally/>
- [^165^] Android 10 Enterprise Features: <https://developer.android.com/work/versions/android-10>
- [^186^] VPN App Store Publishing: <https://klox.app/vpn-app-store-publishing>
- [^187^] iOS Network Extension Security: <https://ptkd.com/journal/ios-network-extension-vpn-entitlement-security>
- [^188^] react-native-wireguard-vpn-connect: <https://www.npmjs.com/package/react-native-wireguard-vpn-connect>
- [^189^] react-native-wireguard-vpn: <https://github.com/usama7365/react-native-wireguard-vpn>
- [^190^] React Native WireGuard Turbo Module: <https://medium.com/@igor.khlyupin/react-native-wireguard-turbo-module-5f2817a24eff>
- [^192^] Secure Mobile Biometric Authentication: <https://blog.ostorlab.co/secure-mobile-biometric-authentication.html>
- [^20^] Mullvad VPN Technical Details: <https://mullvad.net/en/why-mullvad-vpn>

This research document was compiled from 15+ independent web searches across official documentation, GitHub repositories, technical blogs, and community forums. All citations use [^number^] format with source URLs provided in the Sources section.